Slot- c

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA1407	**Course Title:**	COMPILER DESIGN
CO Assessed:	CO1 – Imitation (L1)	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	40% of CIA		
CO1 Statement:	Apply lexical analysis for token specification and recognition using LEX tool. (BL3)		
Student Name:	K. Tharun	**Reg. No.:**	192424265
Section / Batch:	2024	**Date:**	

Code of Conduct

I K. Tharun / 192424265 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. Tharun

Instructions

- Develop a modular and efficient Python program that satisfies all the functional requirements specified in the problem statement.
- Demonstrate the correctness of the implementation using suitable test cases and justify the output obtained.
- Follow good programming practices, including meaningful variable names, proper code indentation, comments, and error handling wherever applicable.
- Duration: 30 minutes.

Q. No	Problem Understanding (20)	Method Selection (20)	Solution Process (25)	Accuracy of Calculation (20)	Interpretation of Results (15)	Total Score (out of 100)
1	4 /4	3 /4	4 /4	4 /4	3 /4	91.25
2	4 /4	3 /4	3 /4	4 /4	4 /4	88.75
3	3 /4	4 /4	3 /4	4 /4	3 /4	85
4	4 /4	4 /4	3 /4	3 /4	3 /4	85
5	3 /4	3 /4	4 /4	4 /4	4 /4	90
OVERALL RUBRICS VALUE						
	/4	/4	/4	/4	/4	88
	/20	/20	/25	/20	/15	/100

N.S.77

Annexure A

ANALYTICAL PROBLEM SOLVING

1	Trace the processing of the input expression P = (a * b) + (b * c) * 2 through all phases of a compiler. a) Identify and list all tokens generated during lexical analysis. b) Construct the syntax tree during the parsing phase. c) Generate the intermediate code for the given expression. d) Apply suitable optimizations to the intermediate code. e) Explain the significance of the final target code prod
2	For each of the following Regular Expressions, construct the language. Write any FIVE valid strings for each Regular Expression. a)$(\varepsilon+aa^*)^*$ b)$(a^*+b^*)^*\ abb$ c)$(a^*b^*)^*\ aba\ (a^*+b^*)^*$ d)$(a+ab)^*$ e) $(aba)^*+(a^*b^*)^*$

3	i)Find the number of tokens in the following C statement is
	main( )
	{
	int a=10;
	char b =”abc”;
	in t c = 30;
	ch ar d = “xyz”;
	in /* comment */ t m = 40.5;
	}
	ii)Identify the lexical errors in the following C statement is
	void main()
	{
	int x=10, y=20;
	char * a;
	a= &x;
	x= 1xab;
	}

4	**Find a regular expression corresponding to each of the following subsets of $\{0, 1\}^*$:** i) The language of all strings that have an even number of 0’s. ii) The language of all strings that contain at least one 1. iii) The language of all strings in which both the number of 0’s and the number of 1’s are odd. iv) The language of all strings that start with 1 and end with 0. v) The language of all strings that do not contain consecutive 1’s.

5	**Consider a lexical analyzer for a simplified programming language. The language consists of the following tokens:** 1. Keywords: if, else, while, return, int, float 2. Operators: +, -, *, /, =, ==, <, > 3. Separators: (,), {, }, ;, , 4. Identifiers: A sequence of letters followed by a sequence of digits, e.g., var123 5. Integer Literals: A sequence of digits, e.g., 12345 6. Floating-point Literals: A sequence of digits followed by a dot and another sequence of digits, e.g., 123.45 **The lexical analyzer needs to tokenize the following input string:** int var123 = 12345; float var456 = 123.45; if (var123 == var456) { return; } **Questions:** a) How many tokens are there in the given input string? b) Provide a list of tokens categorized by their types (keywords, operators, separators, identifiers, integer literals, floating-point literals). c) Assuming a finite state machine (FSM) is used for lexical analysis, estimate the number of state transitions required to tokenize the given input string. Provide your reasoning based on the transitions needed for each token type.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

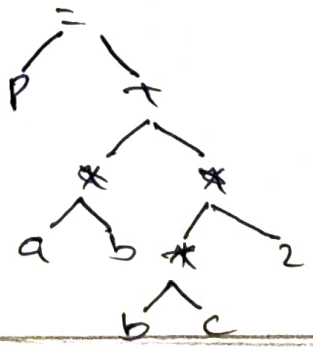
(1) Trace the processing of the expression

expression: p = (a * b) + (b * c) * 2.

a) Tokens generated during lexical analysis

Tokens	→	Type.
1) p	—	identifier.
2) =	—	Assignment operator
3) (	—	Left parenthesis
4) a	—	identifier
5) *	—	multiplication
6) b	—	identifier
7))	—	Right parenthesis.
8) +	—	Addition.
9) c	—	identifier.
10) 2	—	constant.

b) Syntax Tree.



c) Intermediate code.

$t_1 = a * b$

$t_2 = b * c$

$t_3 = t_2 * 2$

$t_4 = t_1 + t_3$

$p = t_4$

d) optimized Intermediate code

$t_1 = a * b$

$t_2 = b * c$

$p = t_1 + (t_2 * 2)$

e) significance of final target code

→ machine code generated for the program

→ executes directly on hardware

→ Faster execution after optimization

→ efficient memory & CPU utilization

(2) Regular expressions - five valid strings

a) $(\varepsilon + aa^*)^*$

Language: Any no. of as

Examples:

→ ε
→ a
→ aa
→ aaa
→ aaaa

b) $(a^* + b^*)^*$ abb

Language: strings ending with abb

Example:

→ abb
→ aabb
→ babb
→ aaabb
→ bbabb

c) $(a^* b^*)^*$ aba $(a^* + b^*)^*$

Language: strings containing aba

Example:

→ aba
→ aaba
→ abab
→ abaaa
→ babab.

d) $(a+ab)^*$

Example:

→ ε
→ a
→ ab
→ aa
→ abab

e) $(aba)^* + (a^*b^*)^*$

Example:

→ ε
→ aba
→ abaaba
→ aaabbb
→ aab.

(3)

i) number of Tokens.

Program:

```
main()
{
int   a=10;
Char  b="abc";
int   c=30;
Char  d="xyz";
int   m=40.5;
}
```

Lexical count:

Statement :- Tokens

main() - 4
{ - 1
int a = 10; - 5
char b = "abc"; - 5
int c = 30; - 5
char d = 'x47'; - 5
int m = 40.5; - 5
} - 1

ii) Lexical errors.

```
void main()
{
int x=10, y=20;
char *a;
a=&x;
x=1xab;
}
```

Errors:

1. 1xab → invalid numeric literal
2. Assigning &x to char *a is a semantic (type error.

④ Regular expression

i) even no. of 0's

$1^*(01^*01^*)^*$

ii) At least one 1

$(0+1)^*1(0+1)^*$

iii) odd no. of 0's & odd no. of 1's

$1^*(01^*01^*)^*01^*(10^*10^*)^*10^*$

iv) start with 1 & ends with 0

$1(0+1)^*0$

v) No consecutive 1's

$(0+10)^*1?$

⑤ Lexical Analyzer

Input:

```
int var123 = 12345;
float var456 = 123.45;
if (var123 == var456)
{
return;
}
```

a) No. of tokens.

Token	Type
int	keyword
var123	identifier
=	operator
1234	integer literal
;	seperator
float	keyword
var456	identifier
=	operator
123.45	floating point
if	keyword
{	seperator
return	keyword
;	seperator

b) Categorized tokens.

Keywords

* int
* float
* if
* return

operator	seperators	identifier	Integer
=	;	var123	123var
=	(	varvar6	
==	)	var123	
	{	varvar6	

c) Estimated FSM state transition

Token type — Approx. tan's

Keywords (4) — 20

Identifier (4) — 24

operator (9) — 4

seperator (7) — 7

Integer literal (1) — 5

Floating literal (1) — 7.